



南京林业大学
NANJING FORESTRY UNIVERSITY

FeHALS：基于 Web 的 3D 可视化航路规划与激光仿真系统 技术设计书

课程名称： “智能 +”GIS 设计与开发

专业： 测绘工程

组别： 7 组

年级： 2023 级

小组成员：	王孜悠	2310604116
	梅康凯	2310604109
	唐如意	2310604115
	杨杰瑞	2310604122
	温璞	2310604117
	刘继明	2110604112
	李英杰	2110604106
	杜仲宇	2310604103
	施祺	2310604113
	向宇鸣	2310604118

指导老师： 隋铭明 那嘉明 陈动 朱杰

批改日期：

提交日期： 2026 年 9 月 8 日

目录

1 背景目的 2

1.1 项目背景 2

1.2 建设目标 2

1.3 设计原则 2

2 需求分析 2

2.1 业务流程 2

2.2 功能需求 2

2.3 非功能需求与约束 2

3 总体设计 3

3.1 系统架构 3

3.2 数据与控制流程 3

3.3 关键设计决策 4

4 功能设计 4

4.1 模型、场景与航点 4

4.2 参数、轨迹和配置 5

4.3 仿真、日志和结果 6

4.4 接口设计 7

5 数据分析 7

5.1 数据对象与组织 7

5.2 坐标、统计与性能 8

6 主要技术手段 8

6.1 前端与三维显示 8

6.2 后端与 HELIOS++ 8

6.3 协作、测试与改进 8

1. 背景目的

1.1. 项目背景

机载激光扫描 (Airborne Laser Scanning, ALS) 能够直接获取地表及地物的三维离散采样, 是数字高程模型生产、森林调查、城市三维建模和灾害监测的重要数据源。真实飞行采集受设备成本、空域、天气和任务风险制约, 外业前通过虚拟场景试验航线与扫描参数, 可以降低反复试飞的成本。HELIOS++ 是面向地面、移动、无人机及机载激光扫描的开源仿真框架, 支持场景、平台、扫描器和测量任务的组合配置^[1], 但其命令行和 XML 配置方式对初学者存在较高门槛。

FeHALS (Frontend of HELIOS++ for Airborne Laser Scanning) 以浏览器为操作入口, 把三维预览、航点编辑、弓字形航线生成、参数配置、HELIOS++ 任务执行、实时日志和点云分析串联为完整流程。开放式 WebGIS 技术便于跨平台访问与后续扩展^{[2][3]}, 浏览器侧利用 WebGL 和 Three.js 完成硬件加速三维显示, 无需安装专用桌面 GIS 软件。

1.2. 建设目标

本项目旨在为 HELIOS++ 提供直观、可交互、可追踪的前端工作台: 集中管理模型、航迹和扫描参数; 在 Z-up 三维场景中完成航点规划; 生成 HELIOS++ 所需轨迹和 XML; 以异步任务运行仿真并实时显示日志; 解析 XYZ、LAS、LAZ 结果, 提供点云渲染与统计; 形成便于多人协作和迭代的 WebGIS 工程结构。

系统面向激光雷达仿真学习者、测绘工程学生和方案验证人员, 定位为教学与原型验证工具, 不替代实际航空作业中的空域审批、飞控、坐标转换及成果质量验收。本设计书以 GitHub 仓库 master 分支当前实现为准, 规划中的 LOD、任务队列、Docker 等功能不会当作现成功能描述。

1.3. 设计原则

系统遵循坐标与参数一致、操作可视化、任务可追踪、前后端模块化和能力渐进增强原则。前端、后端和 HELIOS++ 均以 Z 轴表示高程; 任务具有唯一标识、状态、日志和结果; 组件、状态、接口和服务分层; 当前先满足课程演示和百万点级预览, 超大点云与生产部署作为后续建设内容。

2. 需求分析

2.1. 业务流程

一次完整业务流程为: 启动前后端并检查 HELIOS++ 环境; 上传场景模型并检查方向和包围盒; 手工编辑航点或由矩形生成弓字形航线; 设置平台、飞行与扫描参数; 生成轨迹和配置; 提交仿真并查看实时日志; 加载结果点云、查看统计与直方图; 下载结果或清理缓存。

2.2. 功能需求

系统功能需求如 Table 1 所示。

2.3. 非功能需求与约束

易用性要求核心流程在单页完成, 参数显示单位、范围和默认值; 性能要求全量统计与显示数据分离, 前端渲染最多一百万点, XYZ 每五十万行分块读取; 可靠性要求子进程失败不拖垮 API, 模型异步加载期间删除不产生残留对象; 可维护性要求视图、Pinia 状态、API 客户端、路由、数据模型和业务服务解耦。

表 1: FeHALS 功能需求。

编号	需求说明
FR-01	上传 OBJ、GLTF、GLB、STL；列出、显示/隐藏、删除模型，显示包围盒。仅 OBJ 作为当前 HELIOS++ 场景输入。
FR-02	提供旋转、缩放、平移、视角适配、Z-up 网格和模型方向转换。
FR-03	支持点击添加、拖动、右键或 Delete 删除、列表编辑和清空，并以折线连接。
FR-04	选择矩形范围和航线间距，自动生成连续往返的弓字形航点。
FR-05	以所有模型世界包围盒的最大 Z 加 20m 计算建议高度，并受平台范围约束。
FR-06	设置 UAV/Airborne、速度、高度、扫描频率、扫描半角、脉冲频率和输出格式。
FR-07	生成恒定高度轨迹文件和 HELIOS++ survey XML。
FR-08	异步调用 HELIOS++，记录 pending/running/completed/failed 状态并支持取消。
FR-09	WebSocket 实时推送日志，REST 接口可补取历史日志。
FR-10	查询、下载和解析结果；显示点数、边界、高程/强度统计及 40 组直方图。
FR-11	统计并分类清理 models、trajectories、configs、results。

系统依赖正确安装的 HELIOS++ 及其环境路径，未配置时只能使用三维预览。UAV 与 Airborne 当前均引用 `copter_linearpath` 平台和 `riegl_vux-luav` 扫描器，主要由参数范围区分。航点在 $z = 0$ 地面编辑，生成轨迹时统一叠加飞行高度，属于恒高简化模型。仓库说明记录本机构建的 LASlib 输出异常，因此默认 XYZ；LAS/LAZ 只有在 HELIOS++ 正确链接 LASlib 时可用。任务和配置主要存于单进程内存，系统也未设置账户权限，生产部署前必须补充持久化、鉴权、上传限制、HTTPS 和审计。

3. 总体设计

3.1. 系统架构

FeHALS 采用 B/S 三层架构：Vue 3、Three.js、Pinia 和 Axios 构成表现层；FastAPI、Pydantic、asyncio 与 WebSocket 构成服务层；HELIOS++ 及其平台、扫描器、场景资产构成计算层。本地目录保存模型、轨迹、配置和结果，模型 manifest 用于服务重启后的恢复。前端只交换 JSON、文件和任务标识，后端负责文件组织、配置生成、进程管理和结果解析。

系统总体架构如图 1 所示。

前端中 `Scene3D.vue` 管理画布，`ControlPanel.vue` 管理参数，`WaypointList.vue` 管理航点，`LogConsole.vue` 展示日志，`PointCloudPanel.vue` 展示结果。共享状态由 `scene`、`waypoints`、`simulation` 三个 store 管理；跨组件逻辑放在 `composables`。后端按 API 路由、Pydantic schema、业务 service 和 config 分层。

系统模块架构如图 2 所示。

3.2. 数据与控制流程

组件操作先更新 Pinia，再由 `useHeliosAPI` 请求后端。后端为轨迹、配置和任务分配 UUID，生成文件并启动 HELIOS++ 子进程；日志写入任务缓冲并广播；成功后定位结果文件、计算全量统计并返回显示抽样；前端创建 `THREE.Points` 绘制点云。

系统工作流程如图 3 所示。

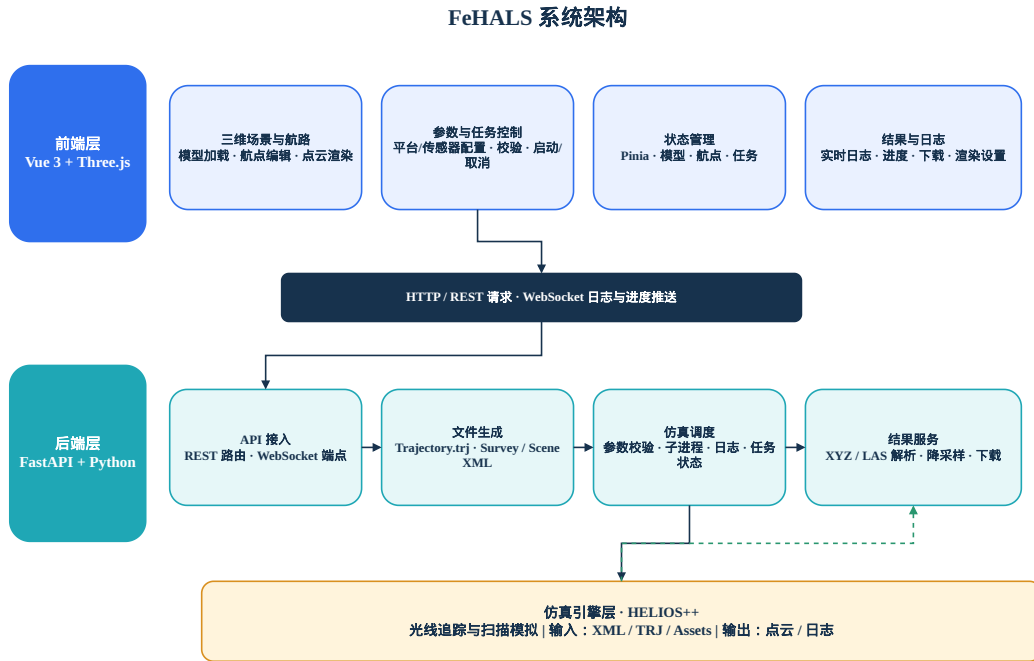


Fig. 1: FeHALS 总体架构

开发环境中 Vite 默认使用 5173 端口，并把/api 和/ws 转发到 8000 端口 FastAPI；后端通过/static 发布文件。Windows 用 run.bat，类 Unix 用 run.sh。当前是单机双进程课程演示结构，尚未包括数据库、反向代理、容器编排和集中监控。

3.3. 关键设计决策

采用 Web 前端可以降低分发门槛并把场景、参数与日志放在同一工作区^[7-9]。系统显式将相机 up 设为 (0,0,1)，Y-up 模型加载时绕 X 轴旋转 90°，保证测绘高程语义一致。资源操作采用 REST，连续日志采用 WebSocket^[2-3]。GLTF/GLB/STL 只承担当下前端预览，避免与 HELIOS++ 输入能力混淆。点云采取“全量统计、抽样渲染”，在统计准确性与交互性能间折中；更大数据仍需要层次 LOD 或渐进渲染^[47]。

4. 功能设计

4.1. 模型、场景与航点

后端校验扩展名并以随机标识保存上传文件，返回静态 URL。前端按扩展名选择 OBJLoader、GLTFLoader 或 STLLoader，将对象加入 modelsGroup。模型可见性和包围盒可切换，Box3 计算尺寸。若模型在异步加载完成前已被删除，pendingRemoval 会使迟到结果被释放，避免“幽灵对象”。相机按包围盒自适应，地面为 XY 平面，Z 为高程。

Raycaster 与 $z = 0$ 隐形地面求交：点击添加航点，拖动修改 XY，右键或 Delete 删除，红色球体和折线表示路径，选中航点变黄。拖动时临时禁用相机控制，以 25 像素平方阈值区分点击和拖动。弓字形航线在矩形内按间距 d 生成，条数为

FeHALS 系统架构图

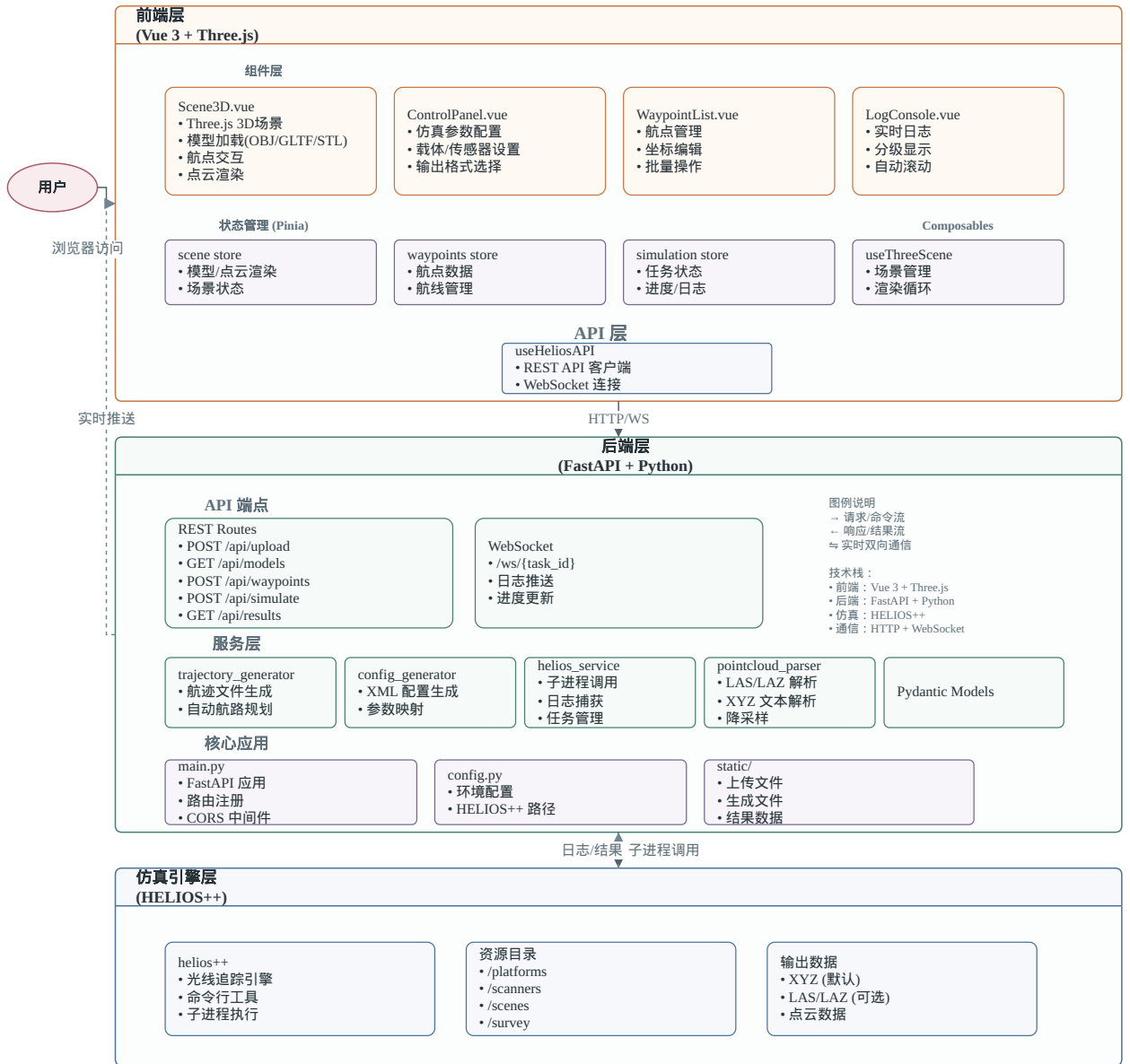


Fig. 2: FeHALS 模块架构

$$N = \left\lceil \frac{y_{\max} * y_{\min}}{d} \right\rceil + 1, \quad (1)$$

相邻扫描线的 X 端点次序交替, 形成连续往返路径。

4.2. 参数、轨迹和配置

建议高度取场景包围盒并集最高点 $z_{\max}$ 加 20 m:



Fig. 3: FeHALS 工作流程

$$h_{\text{suggest}} = z_{\text{max}} + 20 \text{ m.} \quad (2)$$

UAV 速度为 0.5–50 m/s, 高度为 3–500 m; Airborne 速度为 10–200 m/s, 高度为 3–5000 m。RIEGL VUX-1UAV 扫描频率为 10–200 Hz, 扫描半角为 1°–165°, 脉冲频率为 50–550 kHz。后端将航点改写为统一高度的轨迹文件, 并把平台、扫描器、轨迹、速度、频率、半角和格式写入 survey XML。轨迹生成与配置生成分离, 便于复用和排错。

4.3. 仿真、日志和结果

任务提交后检查 trajectory、config 和可选 scene model, 分配 UUID 并由 pending 进入 running。HELIOS++ 标准输出和错误输出逐行保存并广播到 /ws/logs/{task_id}; 退出码为 0 且找到结果时进入 completed, 否则进入 failed。运行中任务可取消。前端保存 taskId、状态、进度、日志和结果, WebSocket 不可用时可通过 REST 查询。

LAS/LAZ 由 laspy 读取^[5], XYZ 分块读取并过滤非法记录。统计包括总点数、XYZ 边界、高程均值、标准差、最小/最大、中位数、P5/P95、40 组高程直方图, 以及存在强度字段时的强度统计。渲染超过一百万点时使用覆盖全文件序列的等间隔索引抽样。前端以 BufferGeometry 和 PointsMaterial 绘制, 支持固定色、高度色和强度色; 归一化为

$$t = \frac{v * v_{\min}}{\max(v_{\max} * v_{\min}, 1)}, \quad (3)$$

再映射至蓝—青—绿—黄—红色带。抽样仅影响显示, 不改变全量统计。

4.4. 接口设计

系统后端 REST 接口如Table 2所示。

表 2: FeHALS 后端接口。

方法	路径	说明
POST	/api/models/upload	上传模型
GET	/api/models	查询模型列表
DELETE	/api/models/{model_id}	删除模型
POST	/api/trajectory/generate	生成航迹
GET	/api/trajectories	查询航迹列表
GET	/api/trajectories/download/{file_id}	下载航迹
POST	/api/config/generate	生成配置
POST	/api/simulation/run	创建仿真任务
GET	/api/simulation/status/{task_id}	查询任务状态
GET	/api/simulation/logs/{task_id}	获取任务日志
POST	/api/simulation/cancel/{task_id}	取消任务
GET	/api/results/{task_id}	获取结果统计和抽样
GET	/api/results/{task_id}/download	下载结果
GET	/api/cache	查看缓存统计
DELETE	/api/cache/{cache_type}	分类清理缓存

主要数据对象如Table 3所示。

表 3: 主要数据对象。

对象	形式	关键字段	用途
场景模型	OBJ/GLTF/GLB/STL	id, name, url, up, bbox	前端均可预览，当前只有 OBJ 进入仿真。
航点	JSON	x, y, z	Pinia 中编辑，生成轨迹前可反复修改。
轨迹	文本	id, altitude, path	由航点生成，供配置引用。
配置	XML	platform, scanner, speed, angle, frequency	供 HELIOS++ 读取。
任务	内存对象	task_id, status, logs, result	当前随服务进程生命周期存在。
结果	XYZ/LAS/LAZ	X, Y, Z, 可选 intensity	保存原始文件并产生统计和显示抽样。

后端用 Pydantic 进行结构校验，资源不存在返回 404，参数或格式错误返回 4xx，子进程失败保留日志。前端在上传、加载、请求和连接处捕获异常并给出可操作提示。公网部署时不应暴露服务器绝对路径和内部堆栈。

5. 数据分析

5.1. 数据对象与组织

后端请求模型包括 TrajectoryRequest、ConfigRequest、SimulationRunRequest 和 SimulationStatus。前端 scene store 管理模型和点云选项，waypoints store 管理航点，simulation store 管理任务、日志、结果与参数。大型 Three.js 对象由单例 useThreeScene 管理，store 只保存业务镜像，避免响应式系统深度代理图形对象。

5.2. 坐标、统计与性能

距离单位为米，速度为 m/s，扫描频率为 Hz，脉冲频率为 kHz，扫描角为半角度数。系统当前不读取 EPSG，也不进行投影转换，因此模型和轨迹必须预先处于同一以米为单位的坐标系；后续应增加 CRS 元数据和 pyproj 转换。

设点数为 n 、高程为 z_i ，则

$$\bar{z} = \frac{1}{n} \sum_{i=1}^n z_i, \quad \sigma_z = \sqrt{\frac{1}{n} \sum_{i=1}^n (z_i - \bar{z})^2}. \quad (4)$$

中位数、P5 和 P95 描述主体分布并降低极端值影响。若 $n > m$ 且 $m = 1,000,000$ ，显示索引可取

$$i_k = \left\lfloor \frac{k(n+1)}{m+1} \right\rfloor, \quad k = 0, \dots, m+1. \quad (5)$$

该方法简单确定，但并非空间均匀采样；超大点云仍需八叉树、多分辨率或渐进策略^[47]。

后端按 models、trajectories、configs、results 建目录。数据依赖为“场景和航点 → 轨迹与参数 → 配置 → 任务 → 点云”。生产版本应记录资源引用，避免清理活动任务依赖。上传侧还需限制大小、校验 MIME 和解析结果、防路径穿越；结果下载只能通过任务记录定位。质量检查至少覆盖航点数、有限坐标、飞行高度、扫描器范围、文件存在、退出码和非空结果。

6. 主要技术手段

6.1. 前端与三维显示

Vue 3 组合式 API 构建单页应用，Vite 提供开发与构建，Pinia 管理共享状态，Axios 统一请求。Three.js 封装相机、光照、材质、Loader、BufferGeometry、Raycaster 和 OrbitControls^[6]，实现模型、航点和点云交互。点云使用单一缓冲几何，避免逐点对象开销。WebGL 无插件硬件加速适合 Web 端地形点云显示^[7]，后续可将解析放入 WebWorker，并引入八叉树 LOD 或渐进渲染^[4]。

6.2. 后端与 HELIOS++

FastAPI 提供异步路由与 OpenAPI 文档^[7]，Pydantic 在业务执行前完成类型校验^[8]，asyncio 子进程避免长任务阻塞 API，WebSocket 负责连续日志^[3]。REST 有利于资源接口解耦^[2]，WebSocket 适合持续消息。HELIOS++ 使用 C++ 实现，支持多类场景、平台和扫描器，并允许在物理真实性和计算成本间配置^[1]。本系统采用命令行子进程集成，便于故障隔离和日志记录；平台/扫描器范围来自 HELIOS++ 资产 XML。

LAS/LAZ 由 laspy 解析^[5]，XYZ 以文本流读取，NumPy 计算统计。Web 端 LiDAR 系统应明确处理、存储和分发边界^[7]；开源 Web 架构可将后端处理与浏览器可视分析组成完整流程^[7]。

6.3. 协作、测试与改进

仓库通过 GitHub 分支、Pull Request 和审查协作；README 说明安装，CHANGELOG 记录演进，TODO 区分已实现与计划项。交付前应补充：轨迹、弓字形、参数边界、XML 转义、XYZ 解析

和统计单元测试；所有 REST 路由的接口测试；小型 OBJ 场景的 HELIOS++ 集成测试；模型增删、航点拖拽、日志重连、点云着色的前端测试；10 万、100 万及更大点云的耗时、帧率和内存测试。

后续应优先完成 HELIOS++ 环境诊断、统一错误结构与自动测试；再将任务、配置和资源关系持久化并引入任务队列；随后增加 CRS、空间索引、LOD、覆盖热力图和仿真动画；最后完成 Docker、反向代理、鉴权、HTTPS、配额与监控。当前 FeHALS 已打通“可视化规划—配置—仿真—日志—点云分析”的核心链路，适合作为课程设计和 HELIOS++ 入门工作台。

参考文献

- [1] WINIWARTER L, Esmorís Pena A M, WEISER H, et al. Virtual laser scanning with HELIOS++: a novel take on ray tracing-based simulation of topographic full-waveform 3D laser scanning[J/OL]. Remote Sensing of Environment, 2022, 269. <https://www.sciencedirect.com/science/article/pii/S0034425721004922>. DOI: <https://doi.org/10.1016/j.rse.2021.112772>.
- [2] FIELDING R T. Architectural styles and the design of network-based software architectures[D/OL]. University of California, Irvine, 2000. <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [3] FETTE I, MELNIKOV A. The WebSocket protocol: 6455[R/OL]. RFC Editor, 2011. DOI: 10.17487/RFC6455.
- [4] SCHÜTZ M, KRONLACHNER M, WIMMER M. Progressive real-time rendering of one billion points[J/OL]. Computer Graphics Forum, 2020, 39(2): 383-394. DOI: 10.1111/cgf.13938.
- [5] MONTAIGU T. laspy: python library for lidar LAS/LAZ IO[EB/OL]. 2017. <https://laspy.readthedocs.io/>.
- [6] CABELLO R. Three.js: JavaScript 3D library[EB/OL]. 2010. <https://threejs.org/>.
- [7] RAMÍREZ S. FastAPI[EB/OL]. 2018. <https://fastapi.tiangolo.com/>.
- [8] COLVIN S. Pydantic: data validation using python type hints[EB/OL]. 2019. <https://docs.pydantic.dev/>.